Projet PPD

Retrieval-Augmented Question Answering

reproduction, analyse critique et évaluations complémentaires

Master Machine Learning pour la Science des Données (MLSD)

Année universitaire 2025–2026

Encadrante : Meriem OUARET

Membres du groupe

Rayane YADEL
Louis VANDAME
Othman ASLOUNE
Franck XU

Table des matières

1 Introduction **1**

2 Article de référence : Retrieval-Augmented Generation **1**
2.1 Problématique du papier . . . 1
2.2 Architecture générale du modèle . . . 1
2.3 Les deux formulations : RAG-Sequence et RAG-Token . . . 2
2.4 Apprentissage et décodage . . . 2
2.5 Principaux résultats reportés dans l'article . . . 3
2.6 Pourquoi cet article est pertinent pour notre projet . . . 3

3 Cadre expérimental du projet **4**
3.1 Données . . . 4
3.2 Organisation du dépôt . . . 4
3.3 Ressources de calcul . . . 4
3.4 Protocoles comparés . . . 4
3.5 Métriques . . . 5

4 Résultats quantitatifs **5**
4.1 Baselines : plafond extractif et No-RAG . . . 5
4.2 RAG-Concat simplifié (naïf) : . . . 6
4.3 Première évaluation manuelle binaire (200 questions) . . . 6
4.4 Effet du corpus indexé : validation vs train . . . 7
4.5 Étude exploratoire du chunking . . . 7

5 Évaluation qualitative : faithfulness et analyse d'erreurs **7**
5.1 Motivation . . . 7
5.2 P1 — Faithfulness . . . 7
5.3 P2 — Error analysis . . . 8
5.4 Comparatif P1 : Qwen 7B vs Qwen 14B (RAG-Concat) . . . 8
5.5 Comparatif P2 : Qwen 7B vs Qwen 14B (RAG-Concat) . . . 8
5.6 Articulation P1/P2 . . . 9
5.7 Proportions des couples P1/P2 . . . 9
5.8 Analyse conjointe . . . 9

6 Extensions : RAG-Concat simplifié vs RAG-Sequence vs RAG-Token **10**
6.1 Positionnement des trois variantes . . . 10
6.2 Résultats automatiques . . . 10
6.3 Point méthodologique important pour RAG-Sequence . . . 11
6.4 Résultats P1 — Faithfulness . . . 11
6.5 Résultats P2 — Analyse d'erreurs . . . 11
6.6 Lecture synthétique . . . 12

7 Discussion générale **12**
7.1 Ce que notre travail confirme . . . 12
7.2 Lien avec l'article de référence . . . 12
7.3 Apports propres du projet . . . 12
7.4 Lecture globale des variantes . . . 12

8 Conclusion **13**

9 Limites et perspectives **13**

1. Introduction

L'*open-domain question answering* consiste à répondre à une question à partir d'un large corpus documentaire, sans que le passage pertinent soit fourni à l'avance. Dans ce cadre, la qualité de la réponse dépend fortement de la capacité du système à retrouver un bon contexte avant l'étape de lecture ou de génération.

Le papier de référence retenu pour ce projet est *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* de Lewis et al. [1]. Cet article propose une approche unifiée combinant :

— une **mémoire paramétrique**, portée par un modèle génératif de type seq2seq ;
— une **mémoire non paramétrique**, constituée d'un index dense de documents récupérés par un retriever.

L'intérêt majeur du papier est de montrer que cette combinaison permet de mieux traiter des tâches dites **knowledge-intensive**, c'est-à-dire des tâches pour lesquelles un modèle ne peut pas raisonnablement répondre sans accès à une source externe de connaissance. L'article introduit notamment deux formulations importantes : **RAG-Sequence** et **RAG-Token**, qui se distinguent par la manière dont elles marginalisent sur les documents récupérés.

Notre travail s'inscrit dans ce cadre général, mais avec un objectif adapté à notre environnement expérimental :

— reproduire l'intuition centrale du RAG ;
— étudier le gain apporté par le retrieval dans un setup contrôlé ;
— comparer plusieurs variantes de RAG avec des LLMs ;
— aller au-delà des métriques automatiques classiques grâce à une analyse qualitative des réponses.

Le rapport est organisé comme suit. Nous commençons par présenter l'article de référence et ses apports principaux, puis nous détaillons notre cadre expérimental et les protocoles comparés. Nous présentons ensuite les résultats quantitatifs, avant de proposer une analyse qualitative centrée sur la *faithfulness* et la catégorisation des erreurs. Enfin, nous discutons les limites du travail et les perspectives.

2. Article de référence : Retrieval-Augmented Generation

2.1. Problématique du papier

Le point de départ de Lewis et al. [1] est le suivant : les grands modèles de langage pré-entraînés stockent une quantité importante de connaissances dans leurs paramètres, mais cette mémoire implicite reste limitée sur plusieurs points :

— difficulté à mettre à jour les connaissances ;
— faible traçabilité des décisions ;
— risque d'hallucination ;
— difficulté à manipuler précisément des connaissances explicites.

L'idée du papier est donc de combiner :

— une **mémoire paramétrique** (un générateur seq2seq pré-entraîné),
— une **mémoire non paramétrique** (un index dense de Wikipédia).

Cette combinaison donne naissance à un modèle de type **Retrieval-Augmented Generation (RAG)**.

2.2. Architecture générale du modèle

Le modèle RAG repose sur deux composants :

— un **retriever** $p_\eta(z \mid x)$, qui retourne une distribution sur des documents z à partir d'une requête x ;

— un **générateur** $p_\theta(y_i \mid x, z, y_{1:i-1})$, qui génère le texte cible en conditionnant sur l'entrée, le document récupéré et les tokens déjà produits.

Dans l'article, le retriever est basé sur **DPR**, tandis que le générateur est un modèle **BART-large**. Le papier utilise un dump Wikipédia de décembre 2018, découpé en chunks de 100 mots, pour constituer la mémoire non paramétrique. Les documents sont indexés de manière dense via **FAISS** [1].

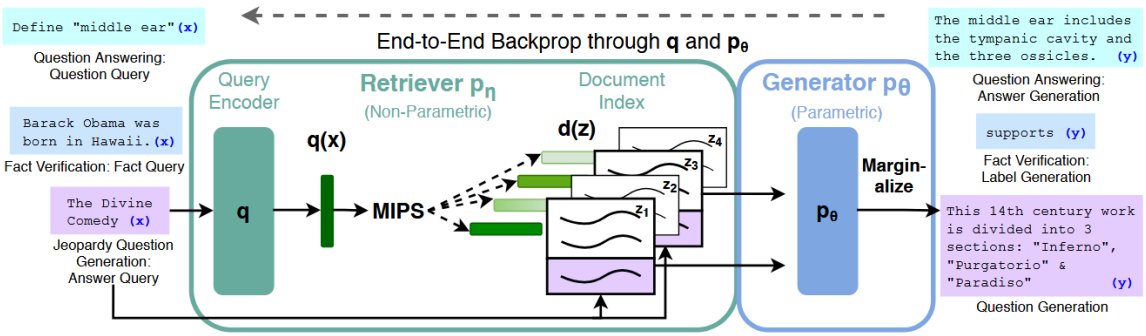

FIGURE 1 – Architecture générale de RAG, reprise de la Figure 1 de [1].

2.3. Les deux formulations : RAG-Sequence et RAG-Token

Le papier introduit deux formulations du modèle.

RAG-Sequence

Dans **RAG-Sequence**, le même document latent est supposé expliquer l'ensemble de la séquence générée. La probabilité de la sortie s'écrit :

$$p_{\text{RAG-Sequence}}(y \mid x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z \mid x)\, p_\theta(y \mid x, z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z \mid x) \prod_{i=1}^{N} p_\theta(y_i \mid x, z, y_{1:i-1}).$$

L'idée est donc qu'un seul document récupéré peut servir de support à toute la réponse.

RAG-Token

Dans **RAG-Token**, la marginalisation se fait au niveau de chaque token :

$$p_{\text{RAG-Token}}(y \mid x) \approx \prod_{i=1}^{N} \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z \mid x)\, p_\theta(y_i \mid x, z, y_{1:i-1}).$$

Cette formulation permet théoriquement au modèle de s'appuyer sur des documents différents selon les tokens générés, donc de combiner plusieurs sources au sein d'une même réponse.

2.4. Apprentissage et décodage

L'article entraîne conjointement le retriever et le générateur à partir de paires entrée/sortie, sans supervision directe sur le document à récupérer. Le document récupéré est traité comme une variable latente, et la perte correspond à la log-vraisemblance marginale négative.

Un point méthodologique important du papier est que :

— le **document encoder** et l'index de documents sont gardés fixes ;
— seuls le **query encoder** et le **générateur** sont fine-tunés.

Au moment du décodage :

— **RAG-Token** peut être décodé comme un modèle autoregressif standard, en marginalisant à chaque étape ;
— **RAG-Sequence** nécessite un décodage plus coûteux, car la probabilité de la séquence ne se décompose pas de manière standard token par token.

2.5. Principaux résultats reportés dans l'article

L'article évalue RAG sur plusieurs tâches **knowledge-intensive** :

— open-domain QA ;
— question answering abstrait (MS-MARCO) ;
— génération de questions de type Jeopardy ;
— fact verification (FEVER).

Pour l'open-domain QA, les résultats reportés montrent que RAG dépasse les approches purement paramétriques et rivalise avec, voire dépasse, plusieurs pipelines spécialisés. Le tableau ci-dessous reprend les principaux scores reportés dans l'article sur quelques benchmarks d'open-domain QA.

Modèle	**NQ**	**WQ**	**CT**
REALM	40.4	40.7	46.8
DPR	41.5	41.1	50.6
RAG-Token	44.1	45.5	50.0
RAG-Sequence	44.5	45.2	52.2

Table 1 – Exemples de résultats d'open-domain QA reportés dans l'article de référence (NQ : Natural Questions, WQ : WebQuestions, CT : CuratedTrec).

L'article montre également que :

— RAG génère des réponses plus **factuelles**, plus **spécifiques** et plus **diverses** qu'une baseline BART purement paramétrique ;
— la mémoire non paramétrique peut être **mise à jour** simplement en remplaçant l'index documentaire, sans réentraîner tout le modèle ;
— les performances dépendent fortement de la qualité du retrieval, ce qui justifie de considérer le retrieval comme une composante centrale du système [1].

2.6. Pourquoi cet article est pertinent pour notre projet

Cet article est particulièrement pertinent pour notre travail pour trois raisons principales :

1. Il fournit un cadre théorique clair pour distinguer **mémoire paramétrique** et **mémoire non paramétrique**.
2. Il introduit explicitement les deux variantes **RAG-Sequence** et **RAG-Token**, qui constituent un prolongement naturel de notre propre pipeline.
3. Il insiste sur le fait que le retrieval ne doit pas être vu comme une étape secondaire, mais comme un levier essentiel de performance, de factualité et de contrôle.

Notre projet ne reprend pas toutes les dimensions du papier à l'identique, mais il s'en inspire directement pour :

— comparer différents régimes de question answering (plafond extractif, No-RAG, RAG) ;
— étudier plusieurs variantes de RAG ;
— analyser la qualité du contexte récupéré et son exploitation par le générateur.

3. Cadre expérimental du projet

3.1. Données

Le dataset utilisé est **SQuAD**. Nous travaillons avec :

— un ensemble **train** d'environ 100 000 exemples ;
— un ensemble **validation** d'environ 10 000 exemples.

Les colonnes principales exploitées dans les fichiers CSV sont : `id`, `title`, `context`, `question`, `gold_answer`.

3.2. Organisation du dépôt

Le dépôt GitHub du projet est disponible ici : github.com/RayaneYadel/RAG.

Il s'organise autour des notebooks suivants :

— `no_rag_QA.ipynb` : baseline QA extractif (RoBERTa SQuAD2), fine-tuning sur train puis inférence sur validation ;
— `no_rag_Qwen.ipynb` : baselines No-RAG avec Qwen 7B et 14B en question seule ;
— `rag_naif.ipynb` : pipeline RAG naïf ;
— `rag_sequence.ipynb` : variante RAG-Sequence ;
— `rag_token.ipynb` : variante RAG-Token ;
— `results/` : métriques, prédictions et fichiers Excel d'annotation.

3.3. Ressources de calcul

Les expériences ont été conduites sur plusieurs environnements :

— **local** : RTX 5060 (8 Go, 60W), plus rapide mais limitée en VRAM ;
— **Onyxia** : NVIDIA A2 (16 Go, 60W), plus de VRAM mais plus lente ;

3.4. Protocoles comparés

Nous avons structuré l'étude autour de plusieurs niveaux d'évaluation :

1. **QA extractif (plafond)** : le modèle reçoit directement le contexte oracle et la question, puis prédit un span.
2. **No-RAG** : le LLM reçoit uniquement la question, sans aucun contexte.
3. **RAG-Concat simplifié (naïf)** : top-k chunks récupérés, concaténés puis injectés en une seule génération.
4. **RAG-Sequence** : génération par chunk/document puis combinaison au niveau séquence.
5. **RAG-Token** : combinaison au niveau token.

Le retriever utilisé dans tous les protocoles RAG est :

`Qwen/Qwen3-Embedding-0.6B`

avec indexation FAISS.

Les principaux paramètres du pipeline RAG sont :

— **top-k** $= 5$;
— **chunking principal** $= 200/50$;
— quelques études complémentaires de chunking.

3.5. Métriques

EM / F1

Nous utilisons les métriques classiques du question answering :

- **Exact Match (EM)** : la réponse générée est considérée correcte uniquement si elle correspond exactement à la réponse de référence après normalisation ;
- **F1 token-level** : cette métrique mesure le chevauchement entre les tokens de la réponse générée et ceux de la réponse de référence, ce qui permet de prendre en compte les réponses partiellement correctes.

Retrieval hit rate

Le **hit rate** correspond au pourcentage de questions pour lesquelles la **gold answer** apparaît dans le contexte récupéré.

Évaluations manuelles

Nous utilisons deux niveaux d'évaluation manuelle :

- une première lecture binaire (réponse correcte ou non) sur 200 exemples ;
- une seconde annotation plus fine, introduisant deux axes :
  - **P1** : *faithfulness*,
  - **P2** : *error analysis*.

4. Résultats quantitatifs

4.1. Baselines : plafond extractif et No-RAG

Bloc	Modèle / setup	EM	F1	Manuel (200)	Temps	GPU
QA (plafond)	RoBERTa SQuAD2	89.9	93.83	–	FT 01 :32 :18 Pred 00 :10 :28	RTX 5060
No-RAG	Qwen 7B	13.6	20.96	29%	00 :10 :21	RTX 5060
No-RAG	Qwen 14B	18.7	27.40	41%	00 :23 :11	RTX 5060

Table 2 – Baselines : plafond extractif et No-RAG.

Le passage du 7B au 14B améliore bien les scores EM/F1 dans le cadre No-RAG, ce qui montre que le modèle 14B dispose d'une mémoire paramétrique plus riche et exploitable que le 7B. Cependant, ce gain reste limité, car l'absence de contexte documentaire impose une contrainte forte : le modèle doit répondre uniquement à partir des connaissances stockées dans ses paramètres, sans possibilité de vérifier, compléter ou mettre à jour l'information au moment de l'inférence. Dans ce cadre, les réponses restent donc plus sensibles à l'oubli, à l'approximation et aux hallucinations.

4.2. RAG-Concat simplifié (naïf) :

Retriever : `Qwen3-Embedding-0.6B` **Chunking :** 200/50 **Top-k** : 5

Bloc	Setup	EM	F1	Hit rate	Temps	GPU
RAG	Retriever 0.6B + Qwen 7B	22.6	37.20	89.9%	05 :40 :26	NVIDIA A2 (16Go, 60W)
RAG	Retriever 0.6B + Qwen 14B	45.8	58.53	89.9%	05 :08 :27	NVIDIA A2 (16Go, 60W)

Table 3 – RAG-Concat simplifié

Le gain apporté par le retrieval est très net par rapport au cadre No-RAG. Sans contexte, les modèles doivent répondre uniquement à partir de leur mémoire paramétrique, ce qui les place dans une situation de "guessing" ou de rappel approximatif. Avec le RAG, ils passent au contraire dans un régime de réponse **basée sur une source**, où un contexte potentiellement pertinent est récupéré avant la génération. Cela se traduit par une amélioration importante des scores : pour Qwen 7B, on passe de **13.6** à **22.6** en EM et de **20.96** à **37.20** en F1 ; pour Qwen 14B, on passe de **18.7** à **45.8** en EM et de **27.40** à **58.53** en F1.

La comparaison entre 7B et 14B en RAG est également instructive. Comme le retriever est identique et que le **hit rate** reste le même (**89.9%**), l'écart observé ne vient pas principalement de la qualité du retrieval, mais de la capacité du LLM à **exploiter le contexte récupéré**. Le 14B utilise mieux les passages fournis, produit des réponses plus précises et plus proches d'un comportement extractif, alors que le 7B bénéficie lui aussi du retrieval mais reste plus limité dans sa capacité à transformer ce contexte en réponse exacte.

4.3. Première évaluation manuelle binaire (200 questions)

En complément des métriques automatiques, une première évaluation manuelle a été réalisée sur un **échantillon fixe de 200 questions**, afin de mieux apprécier la qualité réelle des réponses générées. Le principe était volontairement simple : attribuer **1** si la réponse était jugée globalement correcte, même en présence d'une légère reformulation, et **0** sinon. Cette étape était importante, car dans un cadre génératif, une réponse peut être correcte sans correspondre exactement à la gold answer, ce qui rend la métrique EM parfois trop stricte.

Sur ce même ensemble de 200 questions, les résultats obtenus étaient les suivants :

- **Qwen 7B + RAG** : 143/200 ($\approx 72\%$) ;
- **Qwen 14B + RAG** : 167/200 ($\approx 84\%$).

Cette première lecture manuelle confirme donc la tendance déjà observée avec EM/F1 : l'ajout du retrieval améliore nettement la qualité des réponses, et le **Qwen 14B** exploite mieux le contexte récupéré que le **Qwen 7B**. Elle montre aussi que les performances réelles perçues par lecture humaine peuvent être plus élevées que ce que suggère l'EM strict, en particulier lorsque la réponse est correcte mais reformulée.

Cependant, cette évaluation binaire restait encore limitée : elle permettait de dire si la réponse était globalement correcte ou non, mais pas de comprendre **pourquoi** le système échouait. C'est précisément ce constat qui a motivé l'introduction, dans la suite du projet, d'une analyse qualitative plus fine fondée sur deux axes complémentaires :

- **P1 : Faithfulness**, pour juger si la réponse est effectivement appuyée par le contexte récupéré ;
- **P2 : Error Analysis**, pour distinguer les erreurs dues au retrieval, les erreurs d'exploitation du contexte par le LLM, et les cas où l'EM est trop strict.

4.4. Effet du corpus indexé : validation vs train

Index	EM	F1	Hit rate	Temps	GPU
Validation	43.0	50.68	90.2%	02 :17 :01	RTX 5060
Train	0.7	3.52	15.1%	03 :10 :11	NVIDIA A2

Table 4 – Influence du corpus indexé : validation vs train.

Cette expérience est particulièrement importante : elle montre que le système est aussi bon que la base documentaire qu'il indexe. Lorsque les passages pertinents sont absents, le retrieval s'effondre et les performances chutent brutalement. On observe en outre que, lorsque le contexte manque, le modèle répond souvent `I don't know`, ce qui constitue un comportement intéressant à analyser du point de vue des hallucinations.

4.5. Étude exploratoire du chunking

Chunk	EM	F1	Hit rate	Manuel (200)	Temps	GPU
200/50	63.0	68.98	90.0%	84%	02 :50 :20	NVIDIA A2
80/20	72.3	78.33	85.7%	81%	02 :45 :32	NVIDIA A2
400/100	63.1	68.85	90.7%	86%	02 :57 :54	NVIDIA A2

Table 5 – Étude exploratoire de l'impact du chunking (Qwen 14B).

Cette étude montre que le chunking n'influence pas uniquement le *hit rate* mais aussi la qualité finale du contexte exploité par le LLM. Deux mécanismes peuvent expliquer cela :

— **bruit / dilution du signal** : des chunks trop longs contiennent davantage d'informations non pertinentes ;

— **troncature / saturation du prompt** : lorsque le contexte concaténé devient trop long, une partie utile peut ne plus être transmise correctement au modèle.

Ainsi, un *hit rate* légèrement plus faible peut coexister avec un meilleur EM/F1 si le contexte final est plus compact et moins redondant.

5. Évaluation qualitative : faithfulness et analyse d'erreurs

5.1. Motivation

Les métriques automatiques comme EM/F1 sont indispensables, mais insuffisantes pour caractériser complètement les réponses d'un système génératif. Une réponse peut être :

— correcte mais formulée différemment ;

— partiellement correcte ;

— appuyée par le contexte mais incorrecte par rapport à la gold answer ;

— non justifiée par le contexte récupéré.

Nous avons donc introduit une analyse manuelle plus fine sur un échantillon fixe de 200 questions.

5.2. P1 — Faithfulness

On compare la réponse générée au **contexte récupéré** (top-k chunks concaténés).

Règles :

— **UNSUPPORTED** si la réponse n'est pas justifiée / introuvable dans le contexte récupéré, ou si elle est contredite par le contexte, ou si le modèle répond `I don't know` alors que la gold answer est présente dans le contexte ;
— **PARTIAL** si la réponse est globalement correcte mais ambiguë, vague ou incomplète ;
— **SUPPORTED** sinon.

5.3. P2 — Error analysis

On compare la réponse générée à la **gold answer** et on attribue une cause :

— **EM_OK** : correspondance exacte ;
— **EM_STRICT** : réponse correcte mais reformulée ;
— **RETRIEVER_MISS** : la gold n'apparaît pas dans le contexte récupéré ;
— **LLM_FAIL** : la gold est présente dans le contexte récupéré mais le LLM répond faux ou à côté.

Dans les fichiers Excel des 200 exemples, deux colonnes ont été ajoutées au début :

— **Faithfulness (P1)** : `SUPPORTED / PARTIAL / UNSUPPORTED`
— **Error Analysis (P2)** : `EM_OK / EM_STRICT / LLM_FAIL / RETRIEVER_MISS`

Ces deux colonnes suivent un code couleurs pour faciliter la lecture conjointe et l'analyse.

5.4. Comparatif P1 : Qwen 7B vs Qwen 14B (RAG-Concat)

Classe (P1)	Qwen 7B (%)	Qwen 14B (%)
SUPPORTED	28.5	68.5
PARTIAL	22.0	6.5
UNSUPPORTED	49.5	25.0

Table 6 – Comparatif Faithfulness (P1) — RAG-Concat simplifié, 200 exemples fixes.

Le Qwen 14B est nettement plus fidèle au contexte récupéré que le 7B. À retrieval identique, il produit plus souvent des réponses appuyées par une preuve explicite dans les chunks récupérés, tandis que le 7B dérive plus fréquemment.

5.5. Comparatif P2 : Qwen 7B vs Qwen 14B (RAG-Concat)

Classe (P2)	Qwen 7B (%)	Qwen 14B (%)
EM_OK	12.5	29.5
EM_STRICT	36.0	28.5
LLM_FAIL	41.0	32.0
RETRIEVER_MISS	10.5	10.0

Table 7 – Comparatif Analyse d'erreurs (P2) — RAG-Concat simplifié, 200 exemples fixes.

La part de **RETRIEVER_MISS** reste proche pour les deux modèles, ce qui est cohérent avec un retriever identique. L'écart principal vient donc de l'exploitation du contexte par le LLM.

5.6. Articulation P1/P2

P2 \ P1	SUPPORTED	PARTIAL	UNSUPPORTED
EM_OK	Cas idéal : réponse correcte + preuve explicite	Rare : exact match mais support jugé ambigu	Correct mais sans preuve dans le contexte récupéré
EM_STRICT	Cas idéal : correct + justifié (reformulation)	Correct mais vague/incomplet	Correct mais non justifié par le contexte
LLM_FAIL	Distracteur supporté : le contexte appuie une mauvaise réponse	Partiellement supporté mais incorrect	Erreur/hallucination malgré preuve disponible
RETRIEVER_MISS	Contexte appuie une réponse incorrecte (gold absente)	Support partiel mais gold absente	Gold absente + réponse non justifiée

Table 8 – Lecture conjointe P1/P2 : cas typiques et interprétation.

Lecture rapide. Les couples les plus informatifs sont :

- **SUPPORTED + (EM_OK/EM_STRICT)** : scénario cible (correct et justifié) ;
- **UNSUPPORTED + RETRIEVER_MISS** : retrieval insuffisant ;
- **UNSUPPORTED + LLM_FAIL** : preuve présente mais mauvaise exploitation ;
- **SUPPORTED + LLM_FAIL** : distracteur supporté.

5.7. Proportions des couples P1/P2

P2	P1	Qwen 7B (%)	Qwen 14B (%)
EM_OK	SUPPORTED	12.5	29.0
	PARTIAL	0.0	0.0
	UNSUPPORTED	0.0	0.5
EM_STRICT	SUPPORTED	12.0	27.0
	PARTIAL	13.0	0.5
	UNSUPPORTED	11.0	1.0
LLM_FAIL	SUPPORTED	3.5	11.5
	PARTIAL	9.0	5.5
	UNSUPPORTED	28.5	15.0
RETRIEVER_MISS	SUPPORTED	0.5	1.0
	PARTIAL	0.0	0.5
	UNSUPPORTED	10.0	8.5

Table 9 – Distribution conjointe des couples (P2, P1) — RAG-Concat simplifié, 200 exemples fixes.

5.8. Analyse conjointe

Le gain principal du 14B réside dans la combinaison **"correct + justifié"**. Il obtient une proportion nettement plus élevée de cas **SUPPORTED + (EM_OK/EM_STRICT)** :

$$(29.0 + 27.0) = 56.0\% \quad \text{vs} \quad (12.5 + 12.0) = 24.5\% \text{ pour le 7B.}$$

À l'inverse, le 7B est beaucoup plus dominé par des erreurs non justifiées, notamment le couple **LLM_FAIL + UNSUPPORTED**.

6. Extensions : RAG-Concat simplifié vs RAG-Sequence vs RAG-Token

6.1. Positionnement des trois variantes

Notre implémentation initiale correspond à un **RAG-Concat simplifié** : les top-k chunks récupérés sont concaténés, puis une seule génération est réalisée.

Nous avons ensuite implémenté deux variantes plus proches de l'article de référence :

— **RAG-Sequence** ;
— **RAG-Token**.

Dans cette comparaison, nous conservons :

— le même retriever `Qwen/Qwen3-Embedding-0.6B`,
— le même top-k=5,
— le même chunking 200/50,
— le même LLM Qwen 14B.

6.2. Résultats automatiques

Méthode	EM	F1	Hit rate	Temps	GPU
RAG-Concat simplifié	45.8	58.53	89.9%	05 :08 :27	NVIDIA A2 (16Go, 60W)
RAG-Sequence	39.5	41.16	89.9%	06 :57 :50	NVIDIA A2 (16Go, 60W)
RAG-Token	11.1	14.10	89.9%	02 :04 :43	NVIDIA A2 (16Go, 60W)

Table 10 – Comparatif des trois variantes RAG avec le même retriever et le même LLM.

Le **RAG-Concat simplifié** reste le meilleur des trois sur les métriques automatiques (**EM=45.8**, **F1=58.53**), ce qui montre que, dans notre configuration, la stratégie la plus efficace consiste encore à fournir au générateur l'ensemble des top-k chunks concaténés avant une génération unique. Cette formulation semble offrir le meilleur compromis entre **couverture documentaire** et **simplicité d'exploitation** : le modèle dispose d'un contexte suffisamment large pour contenir la bonne preuve, tout en gardant un mécanisme de génération relativement stable.

RAG-Sequence est en retrait (**EM=39.5**, **F1=41.16**), mais reste nettement au-dessus de **RAG-Token**. Cela suggère que l'idée de raisonner document par document reste pertinente, mais que dans notre implémentation, cette approche perd en efficacité par rapport au RAG-Concat. Une explication plausible est qu'en pratique, conditionner la génération sur un seul chunk à la fois peut rendre le système plus dépendant de la qualité du chunk finalement retenu : si le document choisi n'est pas le meilleur parmi les top-k, la réponse finale en souffre. En revanche, cette approche reste plus robuste que RAG-Token, car elle conserve une cohérence documentaire plus forte au niveau de la séquence entière.

À l'inverse, **RAG-Token** sous-performe fortement (**EM=11.1**, **F1=14.10**). Pourtant, son **hit rate** est identique à celui des deux autres variantes (**89.9%**), ce qui montre clairement que le problème ne vient pas du retrieval brut. Les passages pertinents sont bien récupérés, mais ils sont mal exploités au moment de la génération. Dans cette formulation, le modèle peut théoriquement combiner plusieurs documents au niveau token, mais cette flexibilité semble ici introduire davantage de bruit que de

bénéfice : le générateur peut perdre en stabilité, mélanger des signaux hétérogènes, ou ne pas réussir à maintenir une réponse cohérente sur toute la séquence.

Ainsi, le fait que les trois variantes partagent le même **retrieval hit rate** est un point crucial d'interprétation : l'écart entre elles ne s'explique pas principalement par la capacité du retriever à trouver des passages utiles, mais par la **manière dont ces passages sont ensuite utilisés**. Dans notre setup, le RAG-Concat exploite le mieux le contexte récupéré, RAG-Sequence constitue une alternative intermédiaire intéressante, tandis que RAG-Token apparaît comme la variante la moins robuste.

6.3. Point méthodologique important pour RAG-Sequence

Les trois notebooks utilisent bien le même retriever, le même top-k et le même chunking, et calculent le hit rate au niveau du top-k concaténé.

En revanche, pour le fichier `manual_eval` :

- le **RAG-Concat simplifié** exporte le **contexte top-k concaténé** ;
- le **RAG-Token** exporte aussi le **contexte top-k concaténé** ;
- le **RAG-Sequence** exporte seulement le **chunk sélectionné / utilisé** pour générer la réponse.

Conséquence : dans l'Excel actuel de **RAG-Sequence**, le **RETRIEVER_MISS** annoté ne mesure pas exactement "la gold absente du top-k récupéré", mais plutôt "la gold absente du chunk choisi". Le très gros **RETRIEVER_MISS** observé pour RAG-Sequence ne doit donc pas être compris comme la preuve que le retriever est moins bon. Avec la définition officielle du protocole, le vrai **RETRIEVER_MISS** de RAG-Sequence devrait être beaucoup plus proche de celui des deux autres variantes.

6.4. Résultats P1 — Faithfulness

Classe (P1)	RAG-Concat	RAG-Sequence	RAG-Token
SUPPORTED	68.5	93.5	29.0
PARTIAL	6.5	3.0	3.5
UNSUPPORTED	25.0	3.5	67.5

Table 11 – Comparatif Faithfulness (P1) — Qwen 14B, 200 exemples fixes.

Sur l'axe **faithfulness**, **RAG-Sequence** apparaît comme la variante la plus fidèle au contexte effectivement utilisé, loin devant le RAG-Concat simplifié et surtout RAG-Token. Inversement, **RAG-Token** est massivement **UNSUPPORTED**, ce qui indique que dans une majorité de cas la réponse générée n'est pas correctement appuyée par le contexte.

6.5. Résultats P2 — Analyse d'erreurs

Classe (P2)	RAG-Concat	RAG-Sequence	RAG-Token
EM_OK	29.5	33.0	11.5
EM_STRICT	28.5	6.0	7.0
LLM_FAIL	32.0	2.0	71.0
RETRIEVER_MISS	10.0	59.0	10.5

Table 12 – Comparatif Analyse d'erreurs (P2) — Qwen 14B, 200 exemples fixes.

Plusieurs tendances fortes se dégagent :

— **RAG-Concat simplifié** offre le meilleur compromis global ;
— **RAG-Token** est dominé par **LLM_FAIL**, ce qui indique un problème majeur d'exploitation du contexte ;
— **RAG-Sequence** présente très peu de **LLM_FAIL** et une forte fidélité au chunk utilisé, mais son **RETRIEVER_MISS** brut est artificiellement gonflé par l'export actuel.

6.6. Lecture synthétique

Cette comparaison met en évidence trois profils distincts :

— **RAG-Concat simplifié** : meilleur compromis global entre exactitude, robustesse et exploitation du contexte ;
— **RAG-Sequence** : variante prometteuse, très fidèle au chunk utilisé, plus robuste que RAG-Token, mais nécessitant une réévaluation propre après correction de l'export ;
— **RAG-Token** : variante la plus faible dans notre setup, dominée par des échecs d'exploitation du contexte.

7. Discussion générale

7.1. Ce que notre travail confirme

Notre travail confirme plusieurs idées importantes :

— le retrieval est une composante centrale du système ;
— la qualité du corpus indexé conditionne fortement la performance finale ;
— à retriever identique, une part importante de l'écart entre modèles vient de la manière dont le contexte est exploité ;
— les métriques EM/F1 ne suffisent pas à décrire complètement le comportement du système.

7.2. Lien avec l'article de référence

L'article de Lewis et al. [1] montre que la combinaison d'une mémoire paramétrique et d'une mémoire non paramétrique permet de traiter efficacement des tâches knowledge-intensive. Nos résultats vont dans le même sens :

— l'ajout d'un retriever améliore fortement les performances par rapport au No-RAG ;
— les variantes RAG ne se distinguent pas seulement par le retrieval, mais aussi par leur manière d'intégrer les documents récupérés ;
— l'analyse qualitative est indispensable pour comprendre pourquoi certaines configurations restent faibles malgré un hit rate élevé.

7.3. Apports propres du projet

Par rapport à l'article, notre travail apporte surtout :

— une comparaison structurée entre QA extractif, No-RAG et plusieurs variantes de RAG ;
— une étude de l'impact du corpus indexé ;
— une étude exploratoire du chunking ;
— une évaluation qualitative à deux niveaux (P1/P2) ;

7.4. Lecture globale des variantes

Dans notre configuration expérimentale :

— le **RAG-Concat simplifié 14B** reste la meilleure méthode globale ;
— **RAG-Sequence** semble plus robuste que **RAG-Token** ;

— **RAG-Token** sous-performe fortement, ce qui suggère que son principal problème n'est pas le retrieval brut mais l'exploitation du contexte.

Le cas de **RAG-Sequence** est particulièrement intéressant : même s'il utilise finalement un seul chunk sélectionné, il peut dépasser RAG-Token, ce qui reste cohérent sur une tâche comme SQuAD, où la réponse est souvent courte, localisée et de nature quasi extractive.

8. Conclusion

Ce projet a pris appui sur l'article *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* [1] pour étudier le rôle du retrieval dans un cadre de question answering enrichi par des LLMs génératifs.

Nos expériences montrent clairement que le retrieval joue un rôle structurant dans la qualité des réponses :

— il améliore fortement les performances par rapport au No-RAG ;
— il dépend fortement de la qualité du corpus indexé ;
— il ne suffit pas, à lui seul, à garantir une bonne réponse si le générateur exploite mal le contexte.

Dans notre setup, le **RAG-Concat simplifié 14B** reste la variante la plus performante globalement. Les extensions vers **RAG-Sequence** et **RAG-Token** confirment que la sophistication théorique d'un modèle ne se traduit pas automatiquement par de meilleurs résultats pratiques. Elles montrent surtout que la question centrale n'est pas seulement *quels documents sont récupérés*, mais aussi *comment ils sont exploités* au moment de la génération.

Notre contribution principale au-delà des métriques automatiques réside dans l'introduction d'une analyse qualitative fondée sur la *faithfulness* et la catégorisation fine des erreurs. Cette grille permet de mieux comprendre non seulement *si* le RAG aide, mais aussi *où* il échoue et *pourquoi*.

9. Limites et perspectives

Notre travail présente plusieurs limites :

— le retriever n'a pas été fine-tuné ;
— le cadre expérimental reste centré sur SQuAD ;
— l'évaluation qualitative repose sur 200 exemples ;

Parmi les prolongements possibles :

— comparer plusieurs retrievers à LLM fixé ;
— fine-tuner le retriever ;
— étendre le protocole à d'autres corpus et d'autres datasets ;
— approfondir les métriques de factualité / faithfulness.

Références

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.